

Pearls AQI Predictor - Lahore AQI Predictor

Internship Project Report

1. Project Overview

Pearls AQI Predictor is a Lahore-only machine-learning system that forecasts the US Air Quality Index (AQI) for three future horizons: Day 1 (+24 hours), Day 2 (+48 hours), and Day 3 (+72 hours). The system combines live OpenWeather observations, cloud-backed feature storage, time-series feature engineering, Random Forest forecasting, a separate Model Registry, SHAP explainability, and a Streamlit dashboard.

Area	Current implementation
Supported city	Lahore, Pakistan only
Live source	OpenWeather Air Pollution API and Weather API
Feature Store	Feast with managed PostgreSQL
Model	RandomForestRegressor
Forecast horizons	+24h, +48h, +72h
Explainability	SHAP
Dashboard	Streamlit
Automation	GitHub Actions
Verified tests	20 passed

2. Objectives and Scope

- Collect real Lahore pollutant and weather observations from OpenWeather.
- Calculate numeric US AQI deterministically from PM2.5 using one shared AQI function.
- Store production observations in managed PostgreSQL and serve features through Feast.
- Construct time, lag, and rolling features only from genuine chronological Lahore observations.
- Train separate Random Forest models for +24h, +48h, and +72h forecasting.
- Evaluate forecasts using RMSE, MAE, and R^2 with a chronological train/test split.
- Keep model versions traceable through a separate Model Registry.
- Provide local SHAP explanations and AQI safety alerts in Streamlit.
- Automate hourly feature collection and daily model training using GitHub Actions.

The scope is intentionally limited to Lahore. The fixed coordinates are latitude 31.5204 and longitude 74.3587. No multi-city behavior is claimed in the current implementation.

3. System Architecture

The architecture separates live data collection, cloud storage, feature management, model training, versioning, prediction, and dashboard presentation. Managed PostgreSQL remains the cloud source of truth. Feast provides the offline and online feature-store layer, while the Model Registry is maintained separately for trained model artifacts.

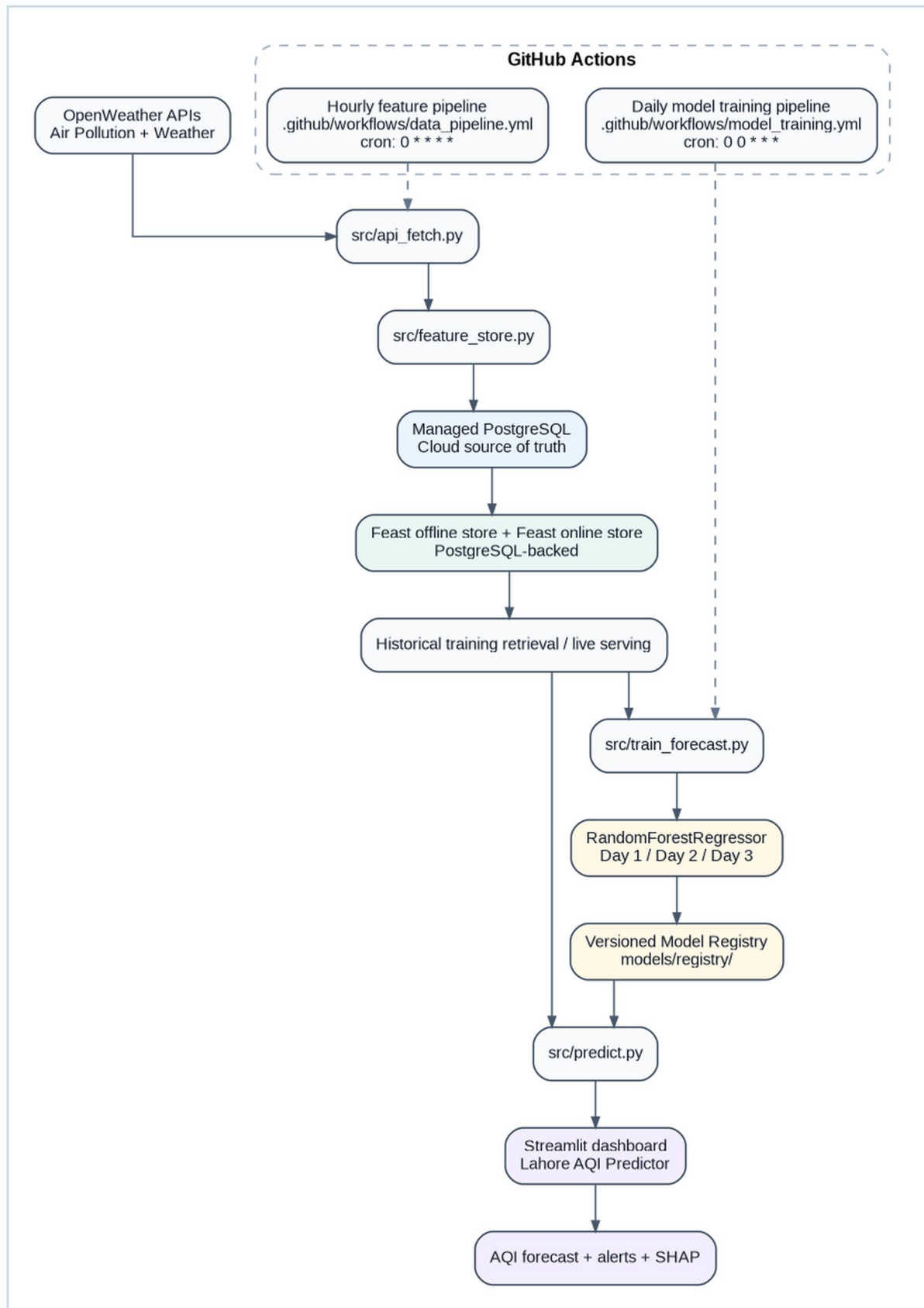


Figure 1. Current Pearls AQI Predictor architecture and GitHub Actions automation.

OpenWeather feeds the application through `src/api_fetch.py` and `src/feature_store.py`. Managed PostgreSQL stores the production history, Feast provides offline/online feature access, `src/train_forecast.py` trains three Random Forest models, `models/registry/` preserves approved versions, and `src/predict.py` serves the Streamlit dashboard. The hourly and daily GitHub Actions workflows operate alongside the main pipeline.

3.1 Hopsworks clarification

The original project brief referenced Hopsworks, but Hopsworks is not used in the final implementation. The Feature Store requirement is fulfilled with Feast plus managed PostgreSQL. Both Feast offline and online stores use PostgreSQL-backed storage, and the production system does not use a local CSV or DuckDB fallback.

4. Data Sources and AQI Calculation

Live data is collected from OpenWeather for the fixed Lahore coordinates. The production pollutant features are PM10, PM2.5, carbon monoxide, nitrogen dioxide, sulphur dioxide, and ozone. Missing pollutant values are not fabricated.

Dataset / source	Verified information
Verified historical EDA dataset	2,184 hourly Lahore observations
Date range	2025-11-06 to 2026-02-04
Missing values	0
Mean AQI	191.8
AQI standard deviation	51.6
Minimum / maximum AQI	60.9 / 383.3
Latest cloud history in dashboard run	7,002 chronological Lahore observations

The repository file `data/historical/air_quality_historical.csv` is intentionally excluded because its Lahore geographic provenance is not verified. This conservative choice prevents unverified records from contaminating training or production history.

4.1 US AQI calculation

Numeric US AQI is calculated deterministically from PM2.5 using the standard EPA breakpoint table. The same shared AQI calculation is reused for historical backfill, live observations, serving, and dashboard alert logic, avoiding inconsistent AQI definitions across the system.

5. Feature Engineering and Shared Feature Contract

Feature type	Features
Raw pollutants	pm10, pm2_5, carbon_monoxide, nitrogen_dioxide, sulphur_dioxide, ozone
Time features	hour, month, day, day_of_week
Lag features	aqi_lag1, aqi_lag2, aqi_lag3
Rolling feature	aqi_rolling_mean

Lag and rolling features are calculated only from real previous chronological Lahore observations. If the available history is insufficient, the system fails closed instead of filling values or inventing history.

`src/feature_contract.py` is the shared source of truth for the feature schema, AQI calculation, timestamp normalization, training feature construction, and serving feature construction. Training and inference therefore use the same feature-generation logic, reducing training-serving skew.

6. Feast Feature Store and Historical Backfill

The Feast project is `pearls_lahore_aqi`. The FeatureView is `lahore_aqi_features` and the entity/join key is `city`. Managed PostgreSQL is the production source of truth, with credentials supplied only through environment variables or secrets.

`src/historical_backfill.py` filters historical data to Lahore, recomputes AQI from PM2.5, validates records, deduplicates by (city, event_timestamp), defaults to dry-run mode, and requires an explicit `--apply` flag before cloud writes. Dry-run-by-default behavior was chosen because historical data provenance required careful validation before production insertion.

7. Training Dataset and Model Training

src/train_forecast.py retrieves chronological Lahore observations from cloud PostgreSQL, resamples them to daily data, constructs separate Day 1, Day 2, and Day 3 training examples, and stops if history is insufficient. The data uses an 80/20 chronological train/test split and is not shuffled, which is appropriate for a forecasting task because future observations must not leak into training.

Random Forest setting	Value
n_estimators	300
min_samples_leaf	2
random_state	42

The active training script trains RandomForestRegressor only. There is no active Ridge comparison and Ridge is not the production model.

8. Model Evaluation

The latest approved model version is 20260906T022717Z. Model quality is reported with RMSE, MAE, and R^2 , and the longer-horizon weaknesses are kept visible rather than being adjusted or hidden.

Forecast horizon	RMSE	MAE	R^2
Day 1 (+24h)	20.0925	16.0665	0.2337
Day 2 (+48h)	22.1753	17.7090	0.0258
Day 3 (+72h)	24.7715	19.7773	-0.2724

Day 1 has modest positive explanatory power. Day 2 is close to zero R^2 , meaning the model explains little variance on that test window. Day 3 has a negative R^2 , which means it performed worse than a mean-prediction baseline on that test window. This is an important current limitation of the project.

9. Model Registry and Three-Day Forecasting

The project keeps trained models in a separate versioned registry under models/registry/. Each version contains day1_model.joblib, day2_model.joblib, day3_model.joblib, and metadata.json. latest.json points to the approved version. The current approved version is 20260906T022717Z.

src/predict.py loads the latest approved version, reconstructs the current daily feature vector from real cloud history, uses the same daily feature construction logic as training, and predicts +24h, +48h, and +72h AQI. Forecasting is restricted to Lahore and is refused when sufficient genuine history is unavailable.

10. Streamlit Dashboard - Actual Run

The following figures are the actual screenshots supplied from the running Streamlit application. They are included as implementation evidence rather than recreated mockups.

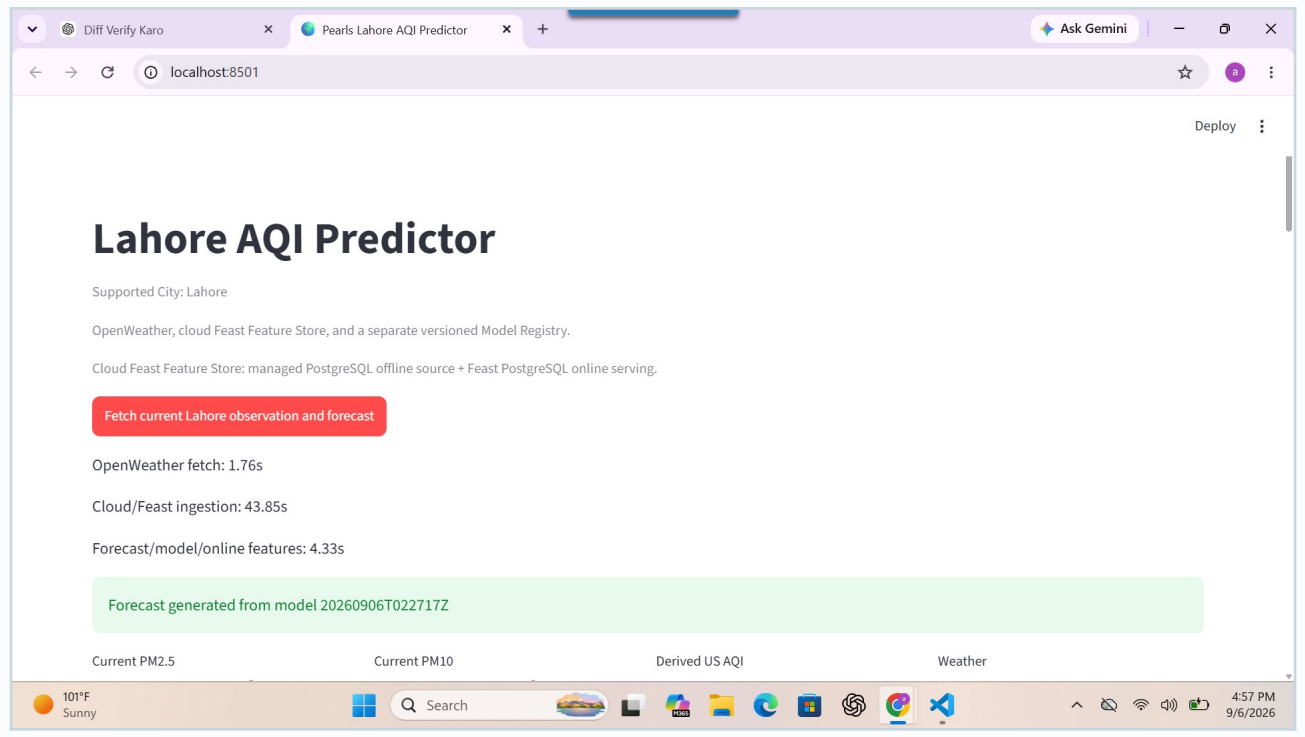


Figure 2. Lahore AQI Predictor dashboard header and live pipeline status.

The dashboard identifies Lahore as the only supported city and states that OpenWeather, the cloud Feast Feature Store, and a separate versioned Model Registry are used. The measured live run shown here recorded 1.76 s for OpenWeather fetching, 43.85 s for cloud/Feast ingestion, and 4.33 s for forecast/model/online-feature processing. The active forecast was produced by model version 20260906T022717Z.

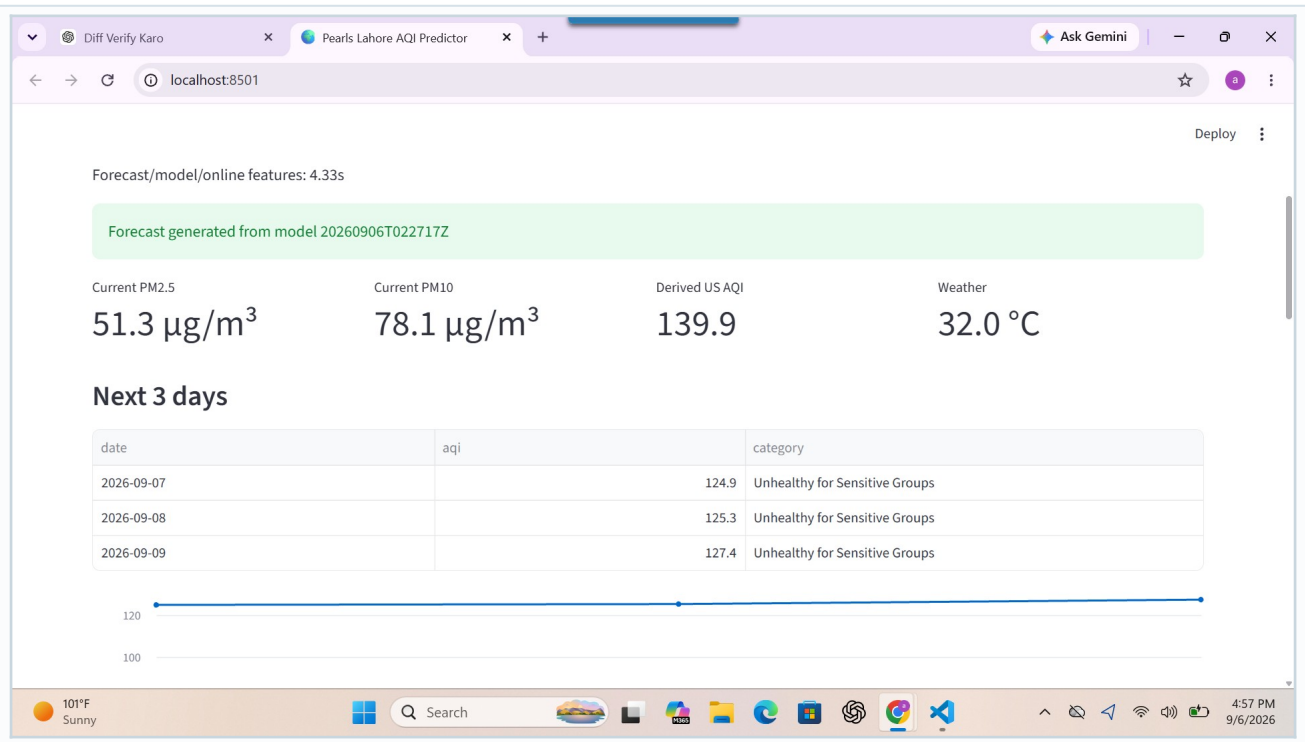


Figure 3. Current Lahore conditions and three-day AQI forecast from the actual dashboard run.

The live dashboard shows PM2.5 = 51.3 µg/m³, PM10 = 78.1 µg/m³, derived US AQI = 139.9, and temperature = 32.0 °C. The displayed three-day forecasts are 124.9, 125.3, and 127.4, all categorized as Unhealthy for Sensitive Groups in this run.

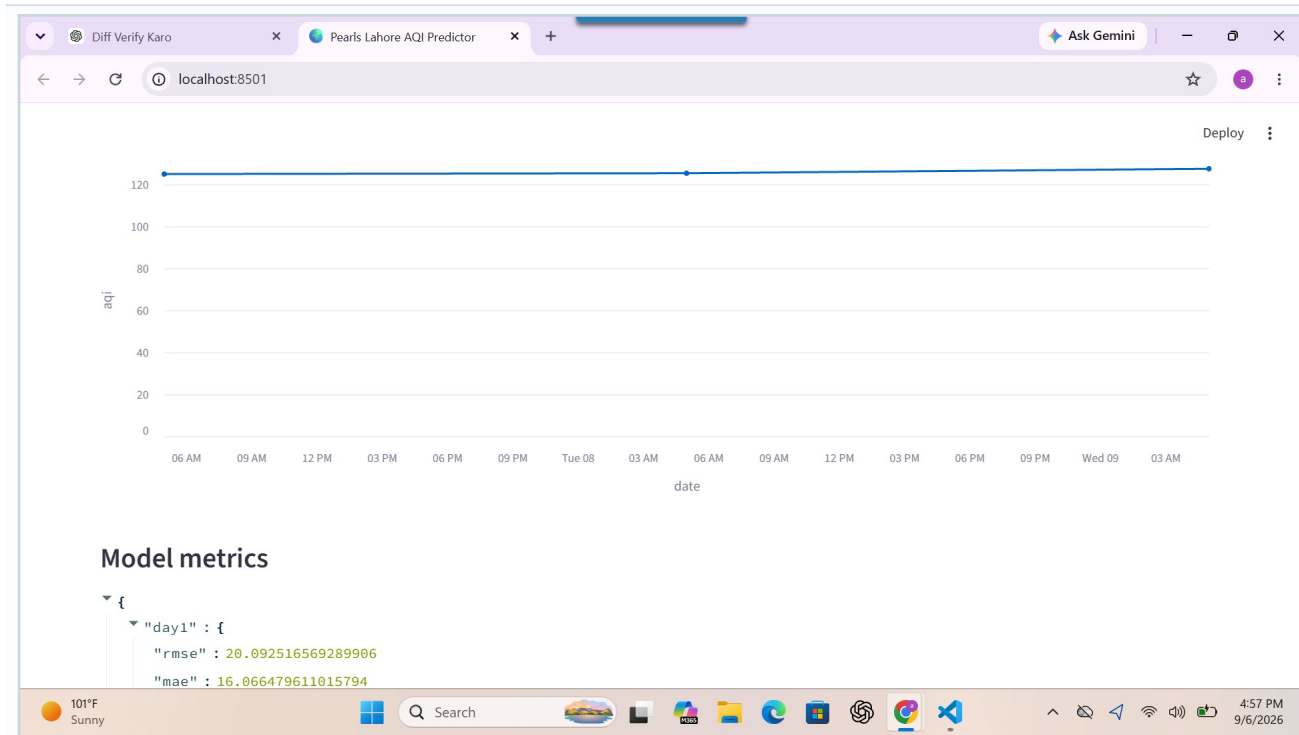


Figure 4. Three-day forecast trend and the start of the model-metrics section.

The forecast line visualizes the three predicted AQI points over the next three days. The values remain in a narrow range in this particular run, while the model-metrics section immediately below exposes the evaluation results rather than presenting the forecast without quality context.

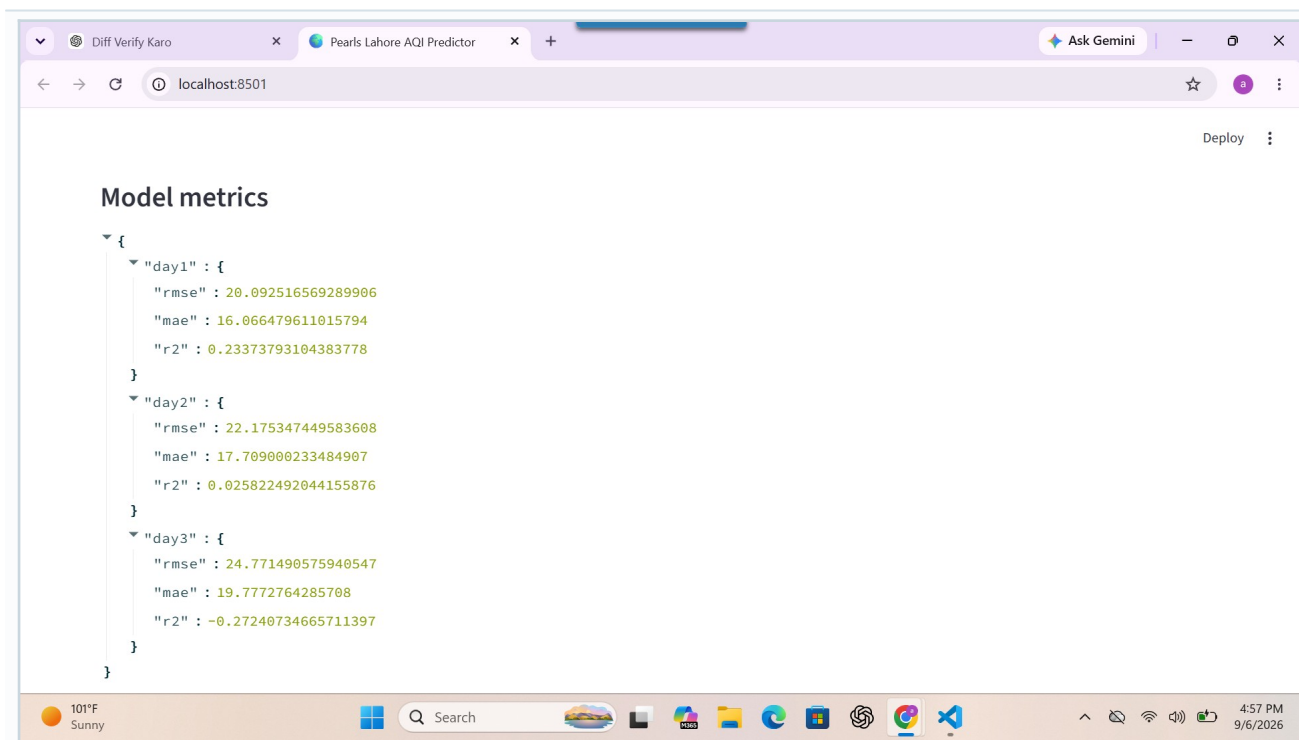


Figure 5. Model evaluation metrics displayed in the Streamlit dashboard.

The dashboard reports the same registered-model metrics used in this report: Day 1 RMSE 20.0925, MAE 16.0665, R^2 0.2337; Day 2 RMSE 22.1753, MAE 17.7090, R^2 0.0258; and Day 3 RMSE 24.7715, MAE 19.7773, R^2 -0.2724. Showing the negative Day 3 R^2 directly supports transparent evaluation.

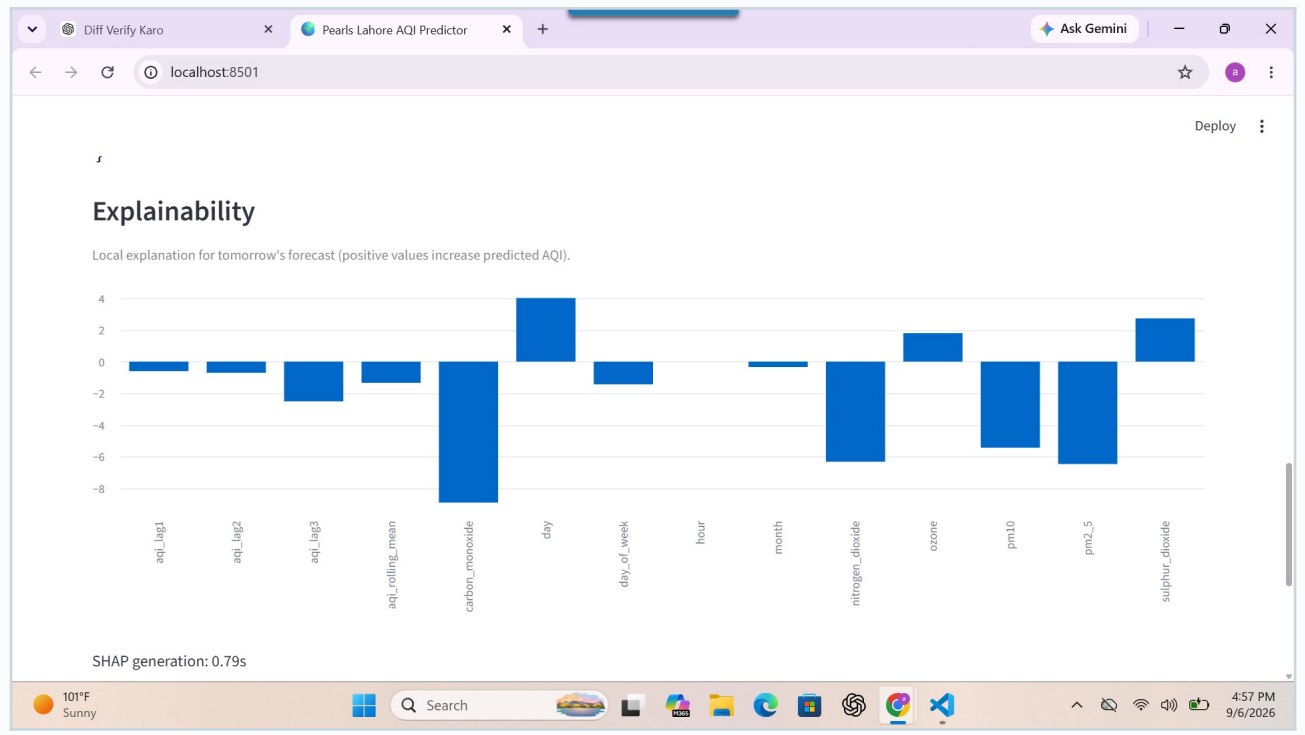


Figure 6. Local SHAP explanation for tomorrow's AQI forecast.

The local SHAP chart shows how individual features move the next-day prediction relative to the model baseline. Positive bars increase the predicted AQI and negative bars decrease it. This figure is an explanation of one specific forecast, not a statement of global causality.

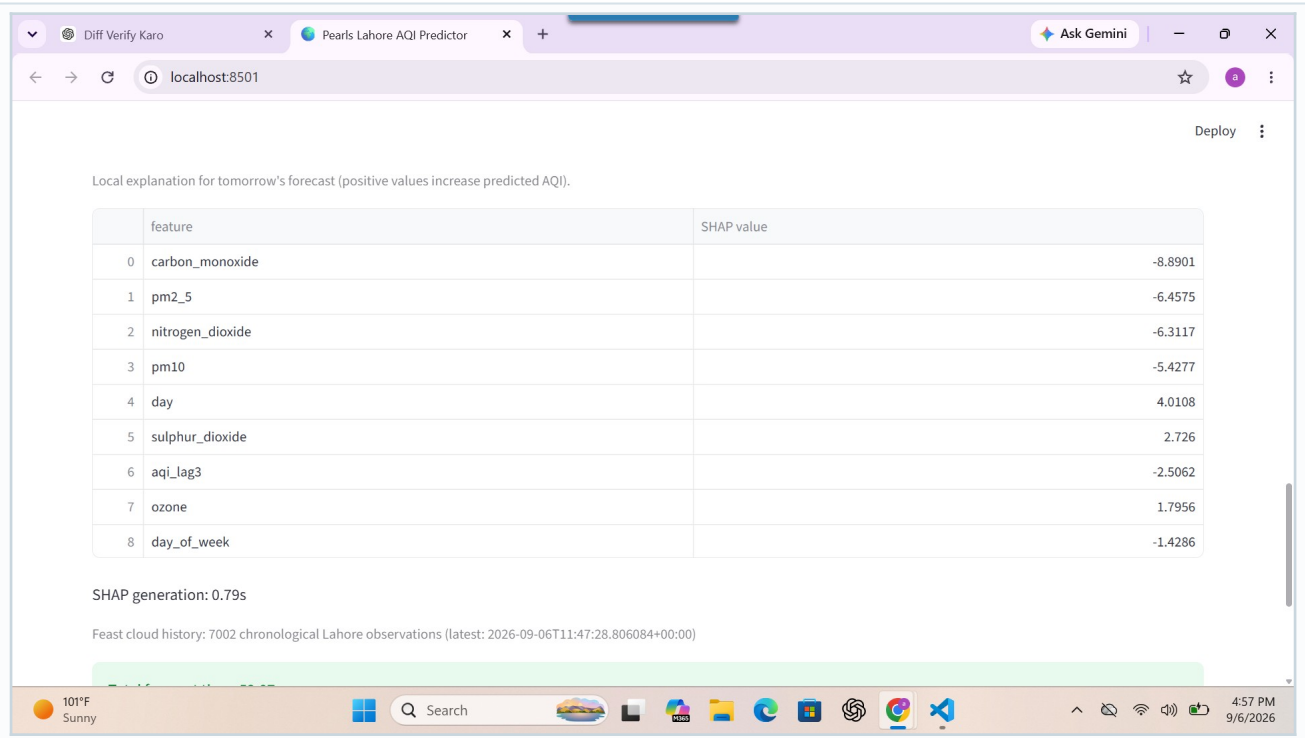


Figure 7. SHAP contribution table and cloud-history status.

The table gives the numerical local SHAP contributions for the same next-day forecast. In the shown run, carbon_monoxide, pm2_5, nitrogen_dioxide, and pm10 have negative local contributions, while day, sulphur_dioxide, and ozone contribute positively. The dashboard also confirms 7,002 chronological Lahore observations and the latest cloud timestamp 2026-09-06T11:47:28.806084+00:00. SHAP generation took 0.79 seconds.

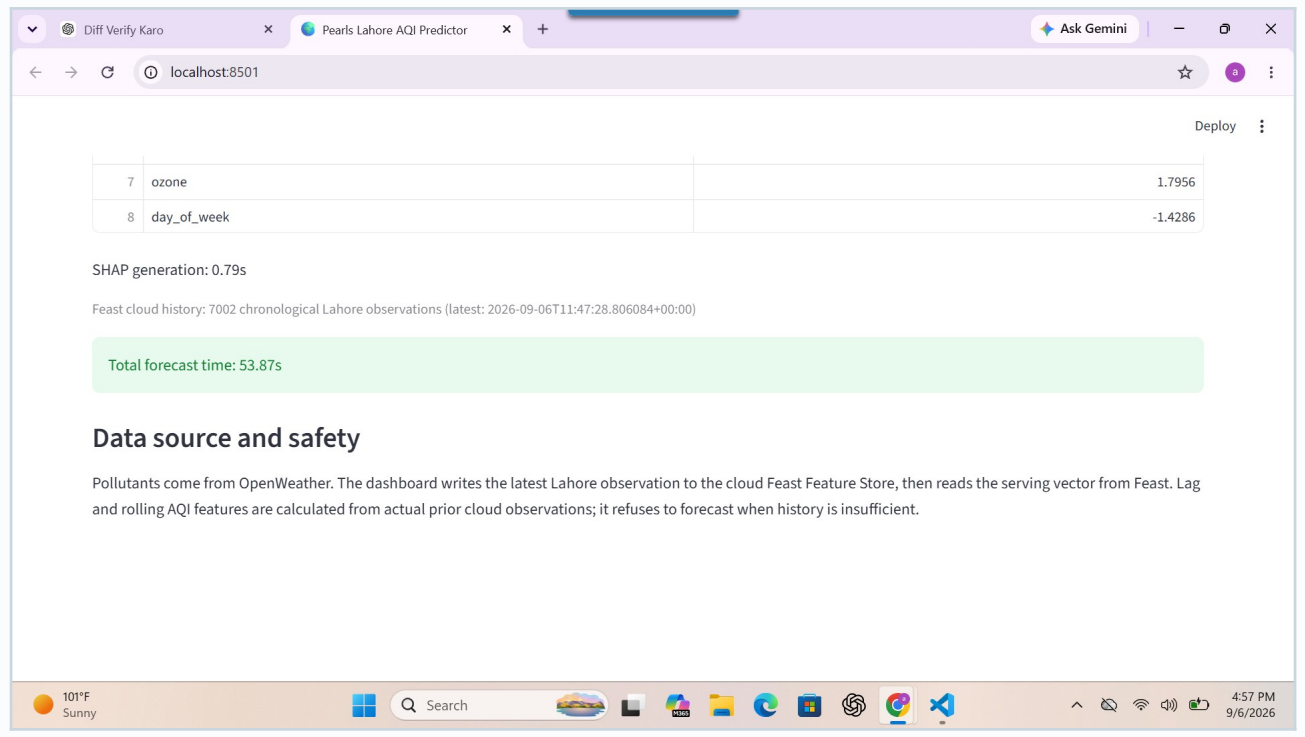


Figure 8. Data-source, safety, and total-runtime information from the actual dashboard.

The lower dashboard section reports a total forecast time of 53.87 seconds and explains the data-safety design: pollutants come from OpenWeather, the latest Lahore observation is written to the cloud Feast Feature Store, lag and rolling features are computed from actual prior cloud observations, and forecasting is refused if history is insufficient.

11. AQI Alerts and Safety Logic

US AQI range	Category
0-50	Good
51-100	Moderate
101-150	Unhealthy for Sensitive Groups
151-200	Unhealthy
201-300	Very Unhealthy
301-500	Hazardous

The dashboard checks both the current AQI and each forecast day against these categories and displays appropriate health warnings. Forecast values above 300 are explicitly highlighted as Hazardous. The current AQI of 139.9 in the shown run falls in the Unhealthy for Sensitive Groups category.

12. SHAP Explainability

src/explainability.py produces global SHAP feature reports during training and local SHAP explanations for dashboard predictions. The dashboard displays the local explanation for tomorrow's forecast. The latest observed SHAP generation time is 0.79 seconds.

13. CI/CD Automation

Workflow	Schedule	Current role
.github/workflows/data_pipeline.yml	0 * * * *	Hourly feature pipeline: checkout, install dependencies, validate secrets, fetch a fresh Lahore observation, and store it in cloud PostgreSQL/Feast.
.github/workflows/model_training.yml	0 0 * * *	Daily model training: checkout, install dependencies, validate secrets, train Day 1/2/3 Random Forest models, generate SHAP reports, upload workflow artifacts, and commit approved registry models back to the repository.

Both workflows also support manual workflow_dispatch execution.

14. Exploratory Data Analysis

EDA is implemented in src/eda.py. The verified dataset contains 2,184 hourly Lahore observations from 2025-11-06 to 2026-02-04 with zero missing values.

AQI statistic	Value
Mean	191.8
Standard deviation	51.6
Minimum	60.9
Maximum	383.3

AQI category	Count
Good	0
Moderate	82
Unhealthy for Sensitive Groups	206
Unhealthy	1,174
Very Unhealthy	632
Hazardous	90

Pollutant	Approx. correlation with AQI
PM2.5	0.98
PM10	0.98
NO2	0.65
CO	0.63
SO2	0.55
O3	-0.52

The very strong PM2.5 correlation is expected because the project derives AQI from PM2.5. The remaining correlations describe associations in this historical sample and should not be interpreted as causal effects.

15. Performance Optimization

A major development issue was slow live ingestion. The original live path loaded too much Lahore history whenever a new observation was processed. The optimized path queries only the recent rows required for live lag and rolling features, with `LIVE_INGESTION_HISTORY_ROWS = 10`. Historical retrieval uses keyset pagination, efficient cloud database access, and incremental Feast materialization while keeping managed PostgreSQL as the source of truth.

Stage	Observed runtime
OpenWeather fetch	1.76 s
Cloud/Feast ingestion	43.85 s
Forecast/model/online features	4.33 s
SHAP generation	0.79 s
Total forecast time	53.87 s

The measured run confirms that cloud/Feast ingestion is the dominant latency component, while model inference and SHAP generation are comparatively small. The optimization improved performance without replacing the required cloud Feature Store architecture or weakening the real-history requirement.

16. Problems Faced and Implemented Solutions

Problem	Implemented solution
Historical data provenance	Excluded the unverified historical file and used only Lahore-validated observations.
Lahore-only geographic validation	Fixed the production coordinates and restricted training, serving, and prediction to Lahore.
Insufficient lag-feature history	Fail closed; do not fabricate lag or rolling values.
Cloud PostgreSQL/Feast performance	Reduced live history reads, used efficient cloud access and keyset pagination.
Incremental Feast materialization overhead	Kept incremental materialization because it preserves the required online feature-store behavior, while reducing unnecessary history reads around it.
Train/serve schema consistency	Centralized schema and feature logic in <code>src/feature_contract.py</code> .
Negative R^2 at longer horizons	Reported the metrics transparently and treated longer-horizon accuracy as a limitation and future improvement area.
Secret management	Used <code>.env</code> locally, GitHub Secrets for CI/CD, and externalized deployment secrets; no credentials are hardcoded.
Model version control	Stored timestamped models and metadata under <code>models/registry/</code> with <code>latest.json</code> pointing to the approved version.

17. Security, Data Safety, and Testing

- No OpenWeather API key is hardcoded.
- No PostgreSQL password is hardcoded.
- `.env` is used locally and is git-ignored.
- GitHub Secrets are used for CI/CD.
- Deployment secrets are externalized.
- The system fails closed when required real data/history is unavailable.
- Unverified historical data is excluded.

The latest verified automated test-suite result is 20 tests passed.

18. Local Setup and Usage

The repository can be prepared and run locally with the following commands:

```
pip install -r requirements.txt
feast -c feature_repo/feature_repo apply
python -m src.feature_pipeline
python -m src.train_forecast
python -m src.eda
streamlit run app.py
pytest -q
```

Required environment variables (values are intentionally not shown):

```
OPENWEATHER_API_KEY
FEAST_POSTGRES_URL
FEAST_POSTGRES_HOST
FEAST_POSTGRES_PORT
FEAST_POSTGRES_DATABASE
FEAST_POSTGRES_SCHEMA
FEAST_POSTGRES_USER
FEAST_POSTGRES_PASSWORD
```

19. Conclusion

Pearls AQI Predictor is a working Lahore-focused internship project that integrates real OpenWeather data, managed PostgreSQL, Feast, shared time-series feature engineering, versioned Random Forest models, SHAP explainability, Streamlit, automated tests, and GitHub Actions. The final implementation does not use Hopsworks and does not claim Ridge or LSTM as active models. The latest registered version is 20260906T022717Z, the verified test result is 20 passed, and the measured end-to-end dashboard run completed in 53.87 seconds.

The project is strongest at the +24-hour horizon and currently has weak longer-horizon performance, including a Day 3 R^2 of -0.2724. Reporting that limitation directly is important because this repository is intended to document the real implementation rather than present artificially improved results.